

Chapter 12A: Matching Game

In our previous chapter we took a deep dive into Windows Forms Applications introducing a number of controls and events. In this chapter we will expand on those concepts by exploring a multi-form application, random number generators and timers.

This chapter is going to expand on an old Microsoft C# tutorial commonly known as the matching game. The basic premise of the matching game is super simple. You have a set of paired images, and you randomly assign them to locations on a playing board. The user has to select pairs of images until s/he has identified the location of each pair of images. As pairs are located, they remain visible on the playing board. If a pair of images are chosen which do not match, they are again hidden after a short period of time. The individual with the shortest overall time to solution is considered the winner.

Cool, let's jump to the inside and learn about multi-form applications, random number generators and timers.

We of course begin by creating a C# Windows Forms Application aptly named Chapter 12 matching game. Our calculator application contained only a single Form, so it wasn't really necessary to rename that guy. Form1 was just fine. This application is going to contain three Forms, so we want them better identified. As such, once your project and default Form are created, please go ahead and rename that Form to UserProfile as this Form is going to be used to capture some basic user demographic information. Finally, please change the Text property of this Form to Matching Game and its Size property to 800,500.

Form1		
	Name	UserProfile
	Text	Matching Game
	Size	800,500

Okay, time to start putting our UserProfile Form together. We begin by adding a title near the top and center of the Form. To do so please open your toolbox and add a Label to our Form. Set the Label's Text property to User Profile and its Font property to bold and 36 point.

So, your title is likely out of center, right? How do you suppose we are going to fix that? Well, the Size property tells us that our Label is 290 pixels wide and 65 pixels tall. We set our Form width to 800 pixels, so we have $800 - 290 = 510$ free pixels across our Form. Half of 510 is 255 so the Location of our Label is going to be 255,x. I played with it a while and I like a value of 255,50 for the Location of our Label1 object.

Label1		
	Text	User Profile
	Font	Segoe UI, Bold and 36pt
	Size	290,65
	Location	255,50

Next please add a second Label and a ComboBox to your form. We will use these to collect the user's salutation so please change the Label's Text property to Salutation: and its Font property to Segoe UI, bold and 12 point. Set the Label's Location property to 200,150.

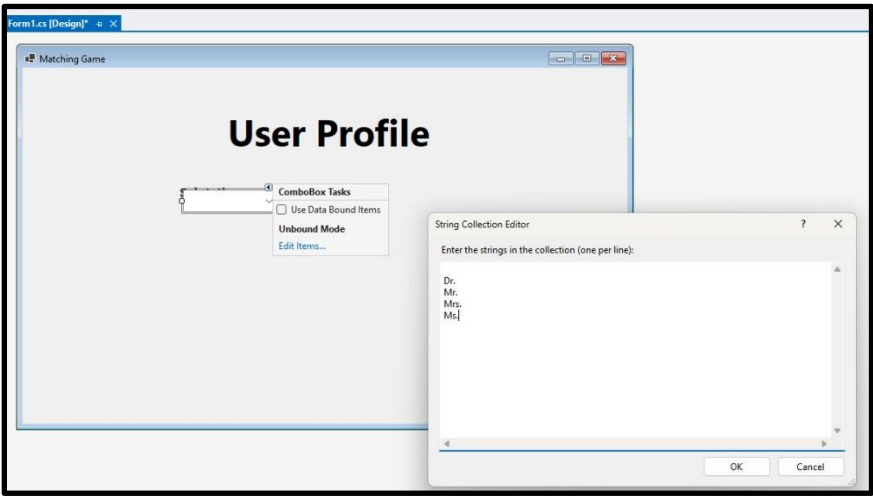
Label2		
	Text	Salutation:
	Font	Segoe UI, Bold and 12pt
	Location	200,150

Begin updating your ComboBox by changing its name to SalutationComboBox. Next change its Font property to Segoe UI, Regular and 12 point. Using the quick menu wizard located at the top right-hand corner of the ComboBox select the Edit Items menu option. Enter each of the items shown in Figure 12A.1. Please be certain to include a blank item as the first (default) entry as Salutation will not be a required field on this form.

Conclude your updates to this ComboBox by setting its Location to 500,150 and its DropDownStyle to DropDownList. This DropDownStyle specifies that the enum list of values will be displayed to the user when they click the arrow and the user is not able to add additional items. The default style is DropDown which also specifies that the list is displayed by clicking the arrow but that the user may enter additional values. If you specify the Simple style the entire list is visible and the user may enter additional values.

ComboBox1		
	Name	SalutationComboBox
	Font	Segoe UI, Regular and 12pt
	Location	500,150
	DropDownStyle	DropDownList

Figure 12A.1: Salutation Items



Build the remainder of your user profile data collection form using the following specifications:
Note, if you want to save some time and possible errors consider simply copying and pasting objects after you have one member of that same class on the form. Also, it is common practice to place an asterisk after required field Labels.

Label3		
	Text	First Name: *
	Font	Segoe UI, Bold and 12pt
	Location	200,200

TextBox 1		
	Name	FirstNameTextBox
	Font	Segoe UI, Regular and 12pt
	Location	500,200
	Size	Use the drag dots and alignment bars to align the right side of this object with the right side of the SalutationComboBox

Label4		
	Text	Last Name: *
	Font	Segoe UI, Bold and 12pt
	Location	200,250

TextBox 2		
	Name	LastNameTextBox
	Font	Segoe UI, Regular and 12pt
	Location	500,250
	Size	Use the drag dots and alignment bars to align the right side of this object with the right side of the SalutationComboBox

Label5		
	Text	Previous Min Time:
	Font	Segoe UI, Bold and 12pt
	Location	200,300

TextBox 3		
	Name	PrevMinTimeTextBox
	Font	Segoe UI, Regular and 12pt
	Location	500,300
	Size	I did a copy/paste so I did not need to readjust
	ReadOnly	True

Please note the ReadOnly property of the next TextBox. If this property is set to true then we can post values to this TextBox via code but no data entry is allowed by the user.

When you set an object's Visible property to False in the properties window it remains visible in the design window but will not be displayed when the application is run. It is common practice to redundantly set that property within the Form's load event handler.

Label6		
	Name	UserErrorMessage
	Font	Segoe UI, Bold and 16pt
	Location	200,425
	Text	First and Last names are required.
	Visible	False

Two more buttons and our form is complete.

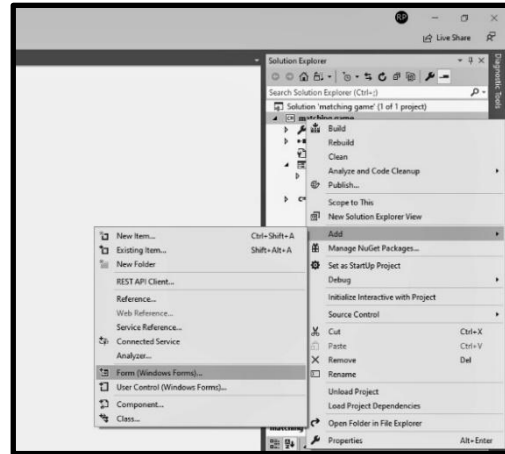
Button1		
	Name	ResetButton
	Font	Segoe UI, Regular and 12pt
	Location	200,360
	Size	150,30
	Text	Reset
	BackColor	Web – Red
	ForeColor	Web - White

Button2		
	Name	SubmitButton
	Font	Segoe UI, Regular and 12pt
	Location	470,360
	Size	150,30
	Text	Submit
	BackColor	Web – Red
	ForeColor	Web - White

At this point, I suggest you run a quick test to ensure that your ComboBox is working properly before we move onto the code behind this guy. By the way, my form looks like that shown here. (Figure 12A.2)

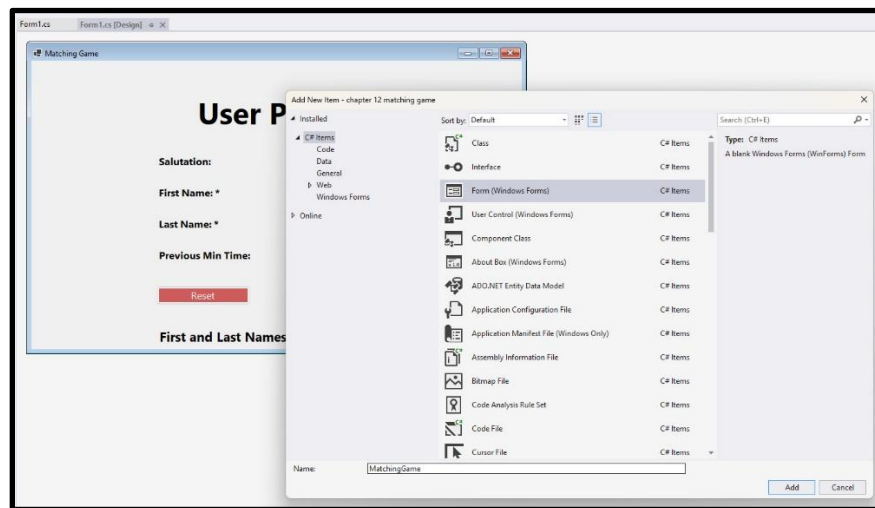
Figure 12A.2: User Profile Form

Okay, before we write the code behind this form let's add a second form to our project. To do this simply go over to the solution explorer pane and right click on the name of the project. From the top-level menu select the Add option. From the submenu select the Form (Windows Forms) option.



The wizard shown here (Figure 12A.3) will be presented. This wizard should already have the Windows Forms option selected. All you need to do is give your new Form a name. Please call it MatchingGame and then hit the Add button.

Figure 12A.3: Add Class Wizard



Visual Studio should now have added a new Form to your design pane with a name of MatchingGame and the Text property also set to MatchingGame. Please change the value of that Text property to Matching Game (two words).

Form2		
	Name	MatchingGame
	Text	Matching Game

We are now ready to write the code behind our UserProfile form. I began by writing the form load code. As I mentioned earlier, it is pretty common practice to redundantly set the Visibility status of our error message label to false during the form load. The `SalutationComboBox.SelectedItem = 0` syntax is of course setting the default value of that ComboBox to the first item on our list, i.e. our blank item. Remember, C# is zero-based. The rest of the code is pretty self-explanatory. (Figure 12A.4)

Figure 12A.4: UserProfile_Load



```
1 namespace chapter_12_matching_game
2 {
3     3 references
4     public partial class UserProfile : Form
5     {
6         1 reference
7         public UserProfile()
8         {
9             InitializeComponent();
10
11         1 reference
12         private void UserProfile_Load(object sender, EventArgs e)
13         {
14             UserErrorMessage.Visible = false;
15             SalutationComboBox.SelectedItem = 0; // blank is default
16             FirstNameTextBox.Text = null;
17             LastNameTextBox.Text = null;
18             PrevMinTimeTextBox.Text = "0";
19         }
20     }
```

I next added two routines named `valid_data` and `SubmitButton_Click`. The `valid_data` routine simply verifies that the first and last name fields are populated and returns a value of true if they are. A value of false is returned if these two fields are not both populated. The `UserErrorMessage` Label is also displayed.

The `SubmitButton_Click` event handler first calls the `valid_data` routine and then opens a new instance of the `MatchingGame` form using the `Show` method if the `valid_data` routine returns a value of true. As the comment indicates, the two forms are then open simultaneously. (Figure 12A.5)

Figure 12A.5: SubmitButton_Click

```
1 reference
private bool valid_data()
{
    if (FirstNameTextBox.Text.Length > 0 && LastNameTextBox.Text.Length > 0) return true;
    else
    {
        UserErrorMessage.Visible = true;
        return false;
    }
}

0 references
private void SubmitButton_Click(object sender, EventArgs e)
{
    if (valid_data())
    {
        UserErrorMessage.Visible = false;

        // instantiate a new MatchingGame form and display simultaneously with UserProfile
        MatchingGame mg = new MatchingGame();
        mg.Show();
    }
    else return;
}
```

The code shown here (Figure 12A.5) results in the two forms being simultaneously open and able to accept focus, i.e. can be the active form. Sometimes multiple, open and active Forms are the preferred design. In other situations, you will want to hide the first Form before opening the second. The syntax for this configuration is shown here. (Figure 12A.6)

Figure 12A.6: Hide UserProfile Form

```
1 reference
private bool valid_data()
{
    if (FirstNameTextBox.Text.Length > 0 && LastNameTextBox.Text.Length > 0) return true;
    else
    {
        UserErrorMessage.Visible = true;
        return false;
    }
}

1 reference
private void SubmitButton_Click(object sender, EventArgs e)
{
    if (valid_data())
    {
        UserErrorMessage.Visible = false;

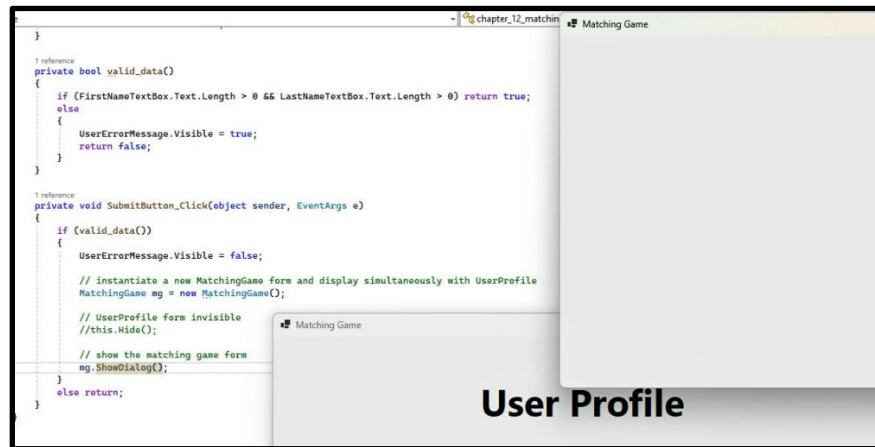
        // instantiate a new MatchingGame form and display simultaneously with UserProfile
        MatchingGame mg = new MatchingGame();

        // UserProfile form invisible
        this.Hide();

        // show the matching game form
        mg.Show();
    }
    else return;
}
```

The image shown here (Figure 12A.7) demonstrates an alternative form of the Show method, the ShowDialog method. The ShowDialog method displays the new Form but requires the focus stay on that new Form. Even if the UserProfile Form is kept visible, the user not able to click on it and return focus to that Form until the MatchingGame Form is closed.

Figure 12A.7: ShowDialog Method



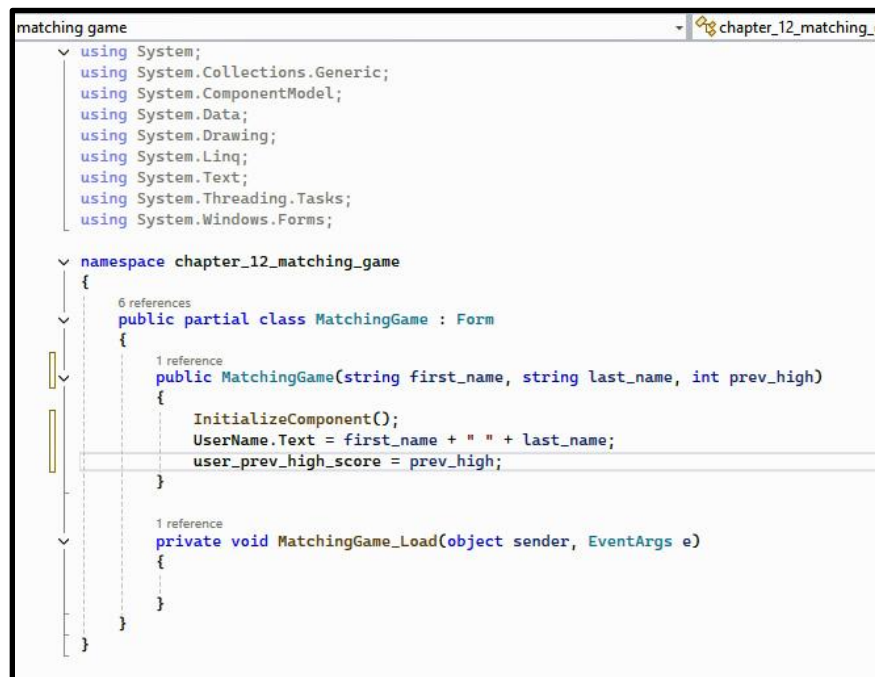
So how do we pass information from the UserProfile Form into the MatchingGame Form? In an earlier chapter we learned how to pass parameters to a class constructor and thus make that information available to the new instance of that class. That is exactly how we are going to pass the user's name to the MatchingGame Form.

We begin by dragging a Label onto the MatchingGame Form. Set the properties of that Label as shown here:

Label1		
	Name	UserName
	Location	20,20
	Font	Segoe UI, Bold, 12 pt

We now move over to the MatchingGame behind file to update the Form's class constructor. (Figure 12A.8) Please note that the `user_prev_high_score` variable is pretty useless in this example, but would have made perfect sense had we been connected to a database or XML file. Don't worry, we will get to that stuff in the next section.

Figure 12A.8: MatchingGame Class Constructor



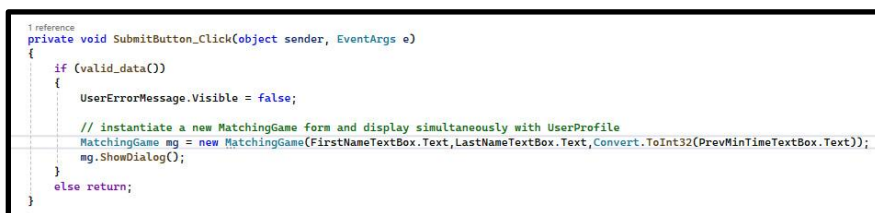
```
matching game
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace chapter_12_matching_game
{
    6 references
    public partial class MatchingGame : Form
    {
        1 reference
        public MatchingGame(string first_name, string last_name, int prev_high)
        {
            InitializeComponent();
            UserName.Text = first_name + " " + last_name;
            user_prev_high_score = prev_high;
        }

        1 reference
        private void MatchingGame_Load(object sender, EventArgs e)
        {
        }
    }
}
```

So, what remains to be completed in order to realize the transfer of information between our UserProfile and MatchingGame Forms? Sure, we need to go back to the submit button on the UserProfile Form and modify our MatchingGame class instantiation code. (Figure 12A.9)

Figure 12A.9: MatchingGame Class Instantiation



```
1 reference
private void SubmitButton_Click(object sender, EventArgs e)
{
    if (valid_data())
    {
        UserErrorMessage.Visible = false;

        // instantiate a new MatchingGame form and display simultaneously with UserProfile
        MatchingGame mg = new MatchingGame(FirstNameTextBox.Text, LastNameTextBox.Text, Convert.ToInt32(PrevMinTimeTextBox.Text));
        mg.ShowDialog();
    }
    else return;
}
```

One more small change before we run a test and call it enough for the first half of this example. After we harvest the information off of our UserProfile Form we really don't need that guy anymore. Therefore, let's save memory and improve performance by hiding that Form after we transfer focus onto the MatchingGame Form. Our next submit code is shown here. (Figure 12A.10) By the way, what would have happened had we replaced the this.Hide syntax with this.Close? Rookie mistake, execution would have halted because no Form would have been open. No worries, everybody makes that mistake when they are first starting out. Use the Hide, ShowDialog sequence and you are always safe.

Figure 12A.10: Hide, ShowDialog Sequence

```
1 reference
private void SubmitButton_Click(object sender, EventArgs e)
{
    if (valid_data())
    {
        UserErrorMessage.Visible = false;
        this.Hide();

        // instantiate a new MatchingGame form and display simultaneously with UserProfile
        MatchingGame mg = new MatchingGame(FirstNameTextBox.Text, LastNameTextBox.Text, Convert.ToInt32(PrevMinTimeTextBox.Text));
        mg.ShowDialog();
    }
    else return;
}
```

Okay, that's about halfway through this example so let's jump to the outside and do a quick review. We will finish up this example in our next lesson.

So, what did we learn so far in this example? We learned how to open multiple forms, pass parameters between the two forms, and how to simultaneously display both forms or force focus onto the second form via the Hide, ShowDialog sequence. Cool stuff.